

第七章：进程间通信

引言

本章导读

在上一章中，我们学习了进程的概念，以及如何使用 `fork`、`exec` 等系统调用来创建新进程。尽管操作系统已经能够方便地动态创建和执行进程，但到目前为止，进程在输入和输出方面仍有许多限制。特别是，进程只能与有限的 I/O 资源交互：它只能读取键盘输入，并将字符输出到屏幕。我们通常将这两者分别称为标准输入和标准输出。此外，进程之间也缺少交换信息的能力，这就限制了它们通过协作来完成更复杂任务的可能性。

其实，在 UNIX 操作系统的早期发展阶段，也遇到了类似的问题。每个程序通常只专注于完成一件特定的事情，但缺乏将多个程序组合起来以实现复杂功能的机制。直到 1975 年，UNIX v6 引入了两个令人眼前一亮的创新机制——I/O 重定向与管道（pipe）。借助这两种机制，操作系统能够在不需要修改应用程序的情况下，将一个程序的输出重新定向到另一个程序的输入中。这样一来，程序之间就可以灵活连接，组合出各种复杂的功能。

在本章中，我们将引入一个新的操作系统概念——管道，并动手实现它。除了键盘和屏幕这样的标准输入与标准输出之外，管道也可以看作一种特殊的输入输出形式。而在后续章节中，我们会讲解文件系统中用于持久化存储数据的抽象——文件，它其实也是一种存储设备上的输入输出。因此，我们可以把标准输入输出、管道和文件都统一在“文件”这个抽象概念之下。这也正体现了 Unix 操作系统中“一切皆文件”（Everything is a file）的重要设计思想。

本章我们会提前引入“文件”这一概念，但并不会详细展开讲解，而是先以最简明的方式对它进行一个初步的设计和实现。从本章操作系统的视角来看，文件是一种需要由操作系统来管理的 I/O 资源。

为了让应用程序能够基于“文件”这个抽象接口进行 I/O 操作，我们就需要对“进程”这个概念进行扩展，使它能够管理文件这种资源。具体来说，需要在进程控制块中增加相应的数据结构。为了统一表示标准输入、标准输出和管道，我们将在每个进程控制块中增设一个文件描述符表，表中保存多条文件记录信息。每个文件描述符是一个非负的整数索引，对应文件记录在文件描述符表中的位置，这样进程就可以方便地引用当前正在使用的标准输入、标准输出和管道（在下一章中，还可以用来表示磁盘上的数据）。用户进程访问文件会变得非常简单：只需通过文件描述符，就能对文件进行读写操作，从而实现从键盘接收输入、向屏幕输出内容，以及在两个进程之间传输数据。

注释：文件描述符就像一个编号，进程通过这个编号来找到对应的文件、管道或标准输入输出，并进行读写。

本章我们的主要目标是实现进程间通信。这意味着进程的输入和输出不一定再指向标准输入输出流（即文件描述符为 0 的 `stdin` 和文件描述符为 1 的 `stdout`），而可能是对应管道的新的文件描述符。考虑到在实验 7 中我们将实现一个较为完整的文件系统，在实验 6 中，我们将借着引入管道的机会，先实现一个文件系统（fs）的雏形。

注释：这样做的目的是为后续的文件系统打下基础，同时让进程通信的机制更加统一和灵活。

实践体验

1. 获取本章代码：

```
$ git checkout ch7
```

2. 在 qemu 模拟器上运行本章代码：

```
$ make test BASE=1
# 在 shell 中执行：
>> ch7b_usertest
```

本章代码树

```
.
├── bootloader
│   └── rustsbi-qemu.bin
├── LICENSE
├── Makefile
├── os
│   ├── console.c
│   ├── console.h
│   ├── const.h
│   ├── defs.h
│   ├── entry.S
│   ├── file.c
│   ├── file.h
│   ├── kalloc.c
│   ├── kalloc.h
│   ├── kernel.ld
│   ├── kernelld.py
│   ├── loader.c
│   ├── loader.h
│   ├── log.h
│   ├── main.c
│   ├── pack.py
│   ├── pipe.c (新增, 管道实现)
│   ├── printf.c
│   ├── printf.h
│   ├── proc.c
│   ├── proc.h
│   ├── riscv.h
│   ├── sbi.c
│   ├── sbi.h
│   ├── string.c
│   ├── string.h
│   ├── switch.S
│   ├── syscall.c
│   ├── syscall.h
│   ├── syscall_ids.h
│   ├── timer.c
│   ├── timer.h
│   ├── trampoline.S
│   ├── trap.c
│   ├── trap.h
│   ├── types.h
│   ├── vm.c
│   └── vm.h
├── README.md
├── scripts
│   ├── kernelld.py
│   └── pack.py
└── user
```

本章代码导读

本章中引入了新的几个系统调用：

```
/// 功能：为当前进程打开一个管道。
/// 参数：pipe 表示应用地址空间中的一个长度为 2 的 long 数组的起始地址，内核需要按顺序将管道读端和写端的文件描述符写入到数组中。
/// 返回值：如果出现了错误则返回 -1，否则返回 0。可能的错误原因是：传入的地址不合法。
/// syscall ID: 59
long sys_pipe(long fd[2]);
/// 功能：当前进程关闭一个文件。
/// 参数：fd 表示要关闭的文件的文件描述符。
/// 返回值：如果成功关闭则返回 0，否则返回 -1。可能的出错原因：传入的文件描述符并不对应一个打开的文件。
/// syscall ID: 57
long sys_close(int fd);
```

同时，为了支持对文件的支持，对 `sys_write` 和 `sys_read` 都有修改。本章的 `pipe` 被我们抽象成了文件的概念，因此，其对应的 `fd` 就是用于 `sys_write` 和 `sys_read` 的 `fd`。我们的 `sys_close` 关闭文件，这本章也就是关闭管道。

文件系统扩充

管道的文件抽象

在 Unix 操作系统出现之前，大多数操作系统采用了复杂且不一致的方式来处理各类 I/O 设备（也叫 I/O 资源），比如键盘、显示器、磁盘这类存储介质，以及串口这类通信设备。这导致应用程序开发变得繁琐，且很难用统一的方法表示和操作 I/O 设备。

随着 UNIX 的诞生，一个简洁而优雅的 I/O 设备抽象被提出来——那就是“文件”。在 Unix 操作系统中，“一切皆文件”是一项重要的设计思想，它继承自 Multics 操作系统的“通用性”理念，并做了进一步的简化。

本章中，操作系统所管理、应用程序所看到的“文件”，本质就是一串字节的集合。操作系统不关心文件的具体内容，只负责提供按字节流进行读写的机制。这意味着任何程序都可以读写任何文件（即字节流），而文件内容的解析完全由应用程序自己负责，操作系统不作干涉。例如，一个 Rust 编译器可以读取 C 语言源文件并进行编译，操作系统不会阻止这种行为。

借助“文件”这一抽象，操作系统内核就能把所有可读写的 I/O 资源当作文件来管理，并将文件分配给进程，让进程通过统一的文件访问接口与各类 I/O 资源交互。

目前我们涉及到的 I/O 硬件设备，大致可分为以下几类：

1. 键盘设备：用于获取字符输入，可抽象为一种只读文件。程序可以从中读取字节序列。
2. 屏幕设备：用于显示字符输出，可抽象为一种只写文件。程序可向其中写入字节序列，内容会直接显示在屏幕上。
3. 串口设备：一种字符通信设备，既能输入也能输出字符，可抽象为可读写文件。程序既可从其中读取字节序列，也可向其中写入要发送的字符。

实际上，也可以将串口设备拆成两个文件：一个用于输入的只读文件和一个用于输出的只写文件。

在 QEMU 模拟的 RISC-V 计算机和 K210 物理硬件上，都配有串口设备。操作系统通过串口设备的输入侧连接到同学使用的计算机键盘，输出侧则连接到计算机的显示器窗口。由于 RustSBI 已经直接管理了串口设备，并为操作系统提供了两个 SBI 接口，因此操作系统可以很方便地通过这两个接口实现字符的输入和输出。

文件是供应用程序使用的，但由操作系统负责管理。虽然文件可以代表多种不同类型的I/O资源，但在进程看来，所有文件的访问都可以通过一个简洁而统一的接口——`File` 结构——来完成。下面我们来看看操作系统框架中对文件结构的扩展：

```
// file.h
struct file {
    enum { FD_NONE = 0, FD_PIPE, FD_INODE, FD_STDIO } type;
    int ref; // reference count
    char readable;
    char writable;
    struct pipe *pipe; // FD_PIPE
    struct inode *ip; // FD_INODE
    uint off;
};
```

pipe管道的实现

管道是一种用于进程间通信的机制，它允许位于管道两端的进程相互传递信息。在我们的操作系统中，管道设计思路十分简单：先寻找一块空闲内存作为管道的数据缓冲区，进程对管道的读写操作就转换为对这块内存区域的读写。

虽然原理看起来直接，但实际读写管道时，进程仍然需要通过系统调用 `sys_write` 和 `sys_read` 来进行。需要注意的是，`sys_write` 还同时负责屏幕输出。此外，一个程序可以拥有多个管道，而管道必须能够让其他程序可见，才能实现进程间通信。为了管理这些功能，每个管道还需要维护一些状态信息，比如上一次读写的位置以及管道中实际可读的数据量等。因此，我们还需要了解操作系统实现管道时的一些具体细节。

首先，我们来看一看管道的结构体。

```
// file.h, 抽象成一个文件了。
#define PIPESIZE 512
struct pipe {
    char data[PIPESIZE];
    uint nread; // number of bytes read
    uint nwrite; // number of bytes written
    int readopen; // read fd is still open
    int writeopen; // write fd is still open
};
```

可以看到，管道把数据存在了一个 `char` 数组的缓存之中来维护。这里我们以ring buffer的形式管理管道的data buffer。

我们来看一下如何创建一个管道。

```
int pipealloc(struct file *f0, struct file *f1)
{
    // 这里没有用预分配，由于 pipe 比较大，直接拿一个页过来，也不算太浪费
    struct pipe *pi = (struct pipe*)kalloc();
    // 一开始 pipe 可读可写，但是已读和已写内容为 0
    pi->readopen = 1;
    pi->writeopen = 1;
    pi->nwrite = 0;
    pi->nread = 0;
```

```
// 两个参数分别通过 filealloc 得到，把该 pipe 和这两个文件关连，一端可读，一端可写。读写端控制是 sys_pipe 的要求。
f0->type = FD_PIPE;
f0->readable = 1;
f0->writable = 0;
f0->pipe = pi;
f1->type = FD_PIPE;
f1->readable = 0;
f1->writable = 1;
f1->pipe = pi;
return 0;
}
```

在操作系统中，我们通常不直接使用“new”来创建结构体，因为我们尚未实现堆内存管理功能。不过，我们可以采用一种相对占用空间但可行的方法：即直接调用 `kalloc()` 申请一整页内存。只要数据结构的大小不超过一个页的大小，就可以用这种方式“创建”出来。

管道（pipe）的两端分别对应输入和输出，在系统中被抽象为两个文件。这两个文件由 `sys_pipe` 系统调用负责创建。在分配管道时，我们会明确设置哪一端用于写入、哪一端用于读取，同时初始化管道内部用于记录读写状态的 `nread` 和 `nwrite` 字段，并建立对缓冲区（buffer）的指针。

注释

这里所说的“new”结构体，是指像高级语言那样动态分配对象。在没有堆内存管理的情况下，我们通过分配整页内存来模拟这一行为，适用于所有不大于一页的结构。

注释

管道两端的读写权限在创建时就已经确定，并通过 `nread` 和 `nwrite` 来跟踪当前读写位置，保证数据能正确在缓冲区中传递。

关闭管道的操作相对简单。实际上，每次关闭只是关闭读写中的一端。只有当读端和写端都被关闭后，系统才会真正释放该管道所占用的资源。

```
void pipeclose(struct pipe *pi, int writable)
{
    if(writable){
        pi->writeopen = 0;
    } else {
        pi->readopen = 0;
    }
    if(pi->readopen == 0 && pi->writeopen == 0){
        kfree((char*)pi);
    }
}
```

重点是管道的读写。

```
int pipewrite(struct pipe *pi, uint64 addr, int n)
{
    // w 记录已经写的字节数
    int w = 0;
    struct proc *p = curr_proc();
    while(w < n){
```

```

// 若不可读，写也没有意义
if(pi->readopen == 0){
    return -1;
}
if(pi->nwrite == pi->nread + PIPESIZE){
    // pipe write 端已满，阻塞
    yield();
} else {
    // 一次读的 size 为 min(用户buffer剩余, pipe 剩余写容量, pipe 剩余线性容量)
    uint64 size = MIN(
        n - w,
        pi->nread + PIPESIZE - pi->nwrite,
        PIPESIZE - (pi->nwrite % PIPESIZE)
    );
    // 使用 copyin 读入用户 buffer 内容
    copyin(p->pagetable, &pi->data[pi->nwrite % PIPESIZE], addr + w, size);
    pi->nwrite += size;
    w += size;
}
}
return w;
}
int piperead(struct pipe *pi, uint64 addr, int n)
{
    // r 记录已经写的字节数
    int r = 0;
    struct proc *p = curr_proc();
    // 若 pipe 可读内容为空，阻塞或者报错
    while(pi->nread == pi->nwrite) {
        if(pi->writeopen)
            yield();
        else
            return -1;
    }
    while(r < n && size != 0) {
        // pipe 可读内容为空，返回
        if(pi->nread == pi->nwrite)
            break;
        // 一次写的 size 为: min(用户buffer剩余, 可读内容, pipe剩余线性容量)
        uint64 size = MIN(
            n - r,
            pi->nwrite - pi->nread,
            PIPESIZE - (pi->nread % PIPESIZE)
        );
        // 使用 copyout 写用户内存
        copyout(p->pagetable, addr + r, &pi->data[pi->nread % PIPESIZE], size);
        pi->nread += size;
        r += size;
    }
    return r;
}
}

```

由于我们的管道是用 ring buffer（环形缓冲区）的形式来管理的，它的容量只有 `PAGESIZE` 大小。因此，我们需要用两个指针 `nread` 和 `nwrite` 来分别记录当前读和写的位置。

这两个指针的值可以大于 `PAGESIZE`，但真正重要的是它们之间的差值（即 `nwrite - nread`），这个差值表示缓冲区中还有多少数据未被读取。

根据管道的工作逻辑，必须先有数据写入，才能进行读取，所以始终满足关系：`nwrite >= nread`。

1. 当 `nwrite == nread` 时，说明缓冲区中已经没有未读的数据了，此时 `piperead` 函数就可以直接退出。
2. 当 `nwrite - nread == PAGESIZE` 时，说明缓冲区已经完全写满，不能再继续写入新数据了，否则会覆盖掉尚未被读取的内容。

如果当前还有空间可以写入，那么就会把数据写入到 `data` 数组中。需要注意的是，因为这是一个环形缓冲区：

如果 `nwrite % PAGESIZE != 0`，并且从当前写入位置到缓冲区末尾的空间不足以放下要写入的全部数据，那么剩余的数据会从缓冲区的开头（即“环头”）继续写入。

大家可以结合 `write` 函数的具体实现代码，仔细体会这一过程。

pipe 相关系统调用

首先是 `sys_pipe`。

```
// os/syscall.c
uint64 sys_pipe(uint64 fdarray) {
    struct proc *p = curr_proc();
    // 申请两个空 file
    struct file* f0 = filealloc();
    struct file* f1 = filealloc();
    // 实际分配一个 pipe，与两个文件关联
    pipealloc(f0, f1);
    // 分配两个 fd，并将之与 文件指针关联
    fd0 = fdalloc(f0);
    fd1 = fdalloc(f1);
    size_t PSIZE = sizeof(fd0);
    copyout(p->pagetable, fdarray, &fd0, sizeof(fd0));
    copyout(p->pagetable, fdarray + sizeof(uint64), &fd1, sizeof(fd1));
    return 0;
}
```

这个系统调用完成了创建一个pipe并记录下两端对应file的功能。并把对应的fd写入user传入的数组地址之中传回user态。

`sys_close` 比较简单。就只是释放掉进程的fd并且清空对应file，并且设置其种类为FD_NONE。

```
uint64 sys_close(int fd)
{
    // 目前不支持 stdio 的关闭，ch7会支持这个
    if (fd <= 2 || fd > FD_BUFFER_SIZE)
        return -1;
    struct proc *p = curr_proc();
    struct file *f = p->files[fd];
    // 目前仅支持关闭 pipe
```

```

    if (f->type == FD_PIPE) {
        fileclose(f);
    } else {
        panic("fileclose: unsupported file type %d fd = %d\n", f->type, fd);
    }
    p->files[fd] = 0;
    return 0;
}

void fileclose(struct file *f)
{
    // ref == 0 才真正关闭
    if(--f->ref > 0) {
        return;
    }
    // pipe 类型需要关闭对应的 pipe
    if(f->type == FD_PIPE){
        pipeclose(f->pipe, f->writable);
    }
    // 清空其他数据
    f->off = 0;
    f->readable = 0;
    f->writable = 0;
    f->ref = 0;
    f->type = FD_NONE;
}

```

原来的 `sys_write` 更名为 `console_write`，新 `sys_write` 根据文件类型分别调用 `console_write` 和 `pipe_write`。`sys_read` 同理。具体的区分是通过判断 `fd` 来进行的。

```

uint64 sys_write(int fd, uint64 va, uint64 len)
{
    if (fd == STDOUT || fd == STDERR) {
        return console_write(va, len);
    }
    if (fd <= 2 || fd > FD_BUFFER_SIZE)
        return -1;
    struct proc *p = curr_proc();
    struct file *f = p->files[fd];
    if (f->type == FD_PIPE) {
        return pipewrite(f->pipe, va, len);
    } else {
        panic("unknown file type %d\n", f->type);
    }
}

uint64 sys_read(int fd, uint64 va, uint64 len)
{
    if (fd == STDIN) {
        return console_read(va, len);
    }
    if (fd <= 2 || fd > FD_BUFFER_SIZE)
        return -1;
    struct proc *p = curr_proc();

```



```

    struct file *f = p->files[fd];
    if (f->type == FD_PIPE) {
        return piperead(f->pipe, va, len);
    } else {
        panic("unknown file type %d fd = %d\n", f->type, fd);
    }
}

```

注意一个文件目前 fd 最大就是15。

进程通讯与 fork

fork的修改

对 fork 的文件支持本来应该在 chapter6 引入，但是为了更好地理解管道的继承机制，我们把它放在了这个章节。

fork 为什么是“毒瘤”呢？因为你总是要在新增一个功能之后，考虑要不要为这个新功能增加 fork 支持。本章的文件系统就是第一个例子。

那么，在 fork 的语境下，子进程也需要继承父进程的文件资源，也就是 PCB（进程控制块）中的文件指针数组。我们应该如何处理呢？

我们来看看 fork 在本 chapter 的具体实现：

注：PCB 中的文件指针数组保存了当前进程打开的所有文件描述符对应的内核数据结构指针。当调用 fork 时，子进程必须获得和父进程一致的文件视图，否则父子进程对同一文件的操作将无法协调，导致行为异常。因此，任何新增的资源管理机制（如管道、特殊设备等），只要能被进程打开或持有，就必须在 fork 时显式处理其继承逻辑。

```

int fork() {
    // ...
    for(int i = 3; i < FD_MAX; ++i)
        if(p->files[i] != 0 && p->files[i]->type != FD_NONE) {
            p->files[i]->ref++;
            np->files[i] = p->files[i];
        }
    // ...
}

```

可以看到创建子进程时会遍历父进程，继承其所有打开的文件，并且给指定文件的 `ref` + 1。因为我们记录的本身就只是一个指针，只需用 `ref` 来记录一个文件还有没有进程使用。

此外，进程结束需要清理的资源除了内存之外增加了文件：

```

void freeproc(struct proc *p)
{
    // ...
    for (int i = 3; i < FD_BUFFER_SIZE; i++) {
        if (p->files[i] != NULL) {
            fclose(p->files[i]);
        }
    }
    // ...
}

```

你会发现 `exec` 的实现竟然没有做任何修改。这是因为 `exec` 仅仅负责重新加载进程要执行的程序镜像（例如新的可执行文件），而不会改变进程的其他属性，比如已经打开的文件。

也就是说，当 `fork` 创建子进程时，子进程会继承父进程当前打开的所有文件（包括管道）；而随后如果子进程调用 `exec`，这些已打开的文件并不会被关闭或“刷掉”。

基于这一点，我们就可以在 `fork` 之前创建好 `pipe`，然后在 `fork` 之后让父子进程中的一方（或双方）调用 `exec`，它们依然能够通过这个 `pipe` 进行进程间通信。

注：`exec` 系列函数只替换进程的代码段、数据段等内容，但保留文件描述符表（除非某个文件描述符设置了 `close-on-exec` 标志）。因此，在 `exec` 之后，原先通过 `fork` 继承下来的 `pipe` 仍然有效，通信通道得以维持。

```

// user/src/ch6b_pipetest
char STR[] = "hello pipe!";
int main() {
    uint64 pipe_fd[2];
    int ret = pipe(&pipe_fd);
    if (fork() == 0) {
        // 子进程，从 pipe 读，和 STR 比较。
        char buffer[32 + 1];
        read(pipe_fd[0], buffer, 32);
        assert(strncmp(buffer, STR, strlen(STR)) == 0);
        exit(0);
    } else {
        // 父进程，写 pipe
        write(pipe_fd[1], STR, strlen(STR));
        int exit_code = 0;
        wait(&exit_code);
        assert(exit_code == 0);
    }
    return 0;
}

```

由于 `fork` 会拷贝父进程的所有文件描述符（即 `fd` 列表），而 `exec` 不会改变已打开的文件描述符（除非文件被标记为 `close-on-exec`），因此父子进程在 `fork` 之后拥有完全一致的 `fd` 列表。这样，它们就可以直接使用在 `fork` 之前创建好的管道（`pipe`）进行通信。

注：文件描述符在 `fork` 后是“复制”而非“共享”——实际上内核中对应的文件对象是共享的，但每个进程拥有自己独立的 `fd` 表项指向同一个内核对象。这意味着父子进程可以通过同一管道读写数据，实现进程间通信。而 `exec` 默认保留这些文件描述符（除非设置了 `FD_CLOEXEC` 标志），所以通常不会影响已建立的管道通信通道。